

Normalization of the Amazon Prime Database

Which tables exist at each stage: UNF → 1NF → 2NF → 3NF → BCNF

1. Overview: the schema grows stage by stage

Normalization splits big, redundant tables into smaller ones. Each stage removes one kind of redundancy, so the **set of tables is different at every stage**. The final result is the 24-table schema in 01_schema.sql.

Stage	Tables	Rule applied	What changed	Tables added
UNF	1	None	One flat PRIME_MASTER table with repeating groups	PRIME_MASTER
1NF	13	Atomic values, no repeating groups, every table has a PK	Repeating groups moved into their own tables	13 flat tables (many still redundant)
2NF	17	No partial dependency on a composite key	3 composite-key tables split	benefit, orders, order_item, product_flat, watchlist, video_content, plan_benefit
3NF	24	No transitive dependency	5 tables split further	plan, seller, category, product, payment_method, payment, episode, music_artist, music_track, ...
BCNF	24	Every determinant is a candidate key	Verification only, no further split	none (final schema)

Number of tables: **1 → 13 → 17 → 24 → 24**

2. Key terms

- **Functional dependency (FD):** $A \rightarrow B$ means knowing A gives exactly one value of B (e.g. $\text{plan_id} \rightarrow \text{price}$).
- **Partial dependency (2NF problem):** a column depends on only *part* of a composite key, e.g. product_name depends on product_id but the key is $(\text{order_id}, \text{product_id})$.
- **Transitive dependency (3NF problem):** a non-key column determines another non-key column, e.g. $\text{product_id} \rightarrow \text{seller_id} \rightarrow \text{seller_city}$.
- **BCNF:** a stricter 3NF. Every determinant (left side of an FD) must be a candidate key.

3. Starting point: the unnormalized form (UNF)

```
PRIME_MASTER(customer_id, full_name, email, phone, {addresses: line1, city, state, pincode}, {profiles},  
plan_name, plan_price, {benefits}, {orders: order_id, order_date, status, {items: product_id, product_name,  
product_price, seller_name, seller_city, category_name, quantity}}, {payments: method_type, provider,  
amount}, {watch: content_id, title, episode}, {music plays: track, artist})
```

Braces { } mark repeating groups. Problems: a seller's city is repeated on every item row (update anomaly), a product cannot exist until it is ordered (insert anomaly), and deleting the last order deletes the product and seller (delete anomaly).

4. First Normal Form (1NF): 13 tables

Rule: every column holds one value, there are no repeating groups, and every table has a primary key. **How:** each repeating group (addresses, profiles, orders, items, payments, plays) becomes its own table linked by a foreign key. The tables are still redundant, which is fixed in later stages.

Table	Primary key	Columns	Status	Remaining problem
customer	customer_id	customer_id, full_name, email, phone, password_hash, date_of_birth, created_at	NEW	Already atomic
address	address_id	address_id, customer_id, label, line1, city, state, pincode, country, is_default	NEW	'Many addresses' group moved out
profile	profile_id	profile_id, customer_id, profile_name, is_kids	NEW	'Many profiles' group moved out
subscription_flat	subscription_id	subscription_id, customer_id, plan_id, plan_name, plan_price, duration_months, max_profiles, has_ads, start_date, end_date, status, auto_renew	NEW	Plan details repeated on every subscription
subscription_audit	audit_id	audit_id, subscription_id, old_status, new_status, changed_at	NEW	Status history group moved out
plan_benefit_flat	(plan_id, benefit_id)	plan_id, benefit_id, benefit_name, benefit_description	NEW	Multi-valued 'benefits' split into rows
order_line_flat	(order_id, product_id)	order_id, product_id, customer_id, address_id, order_date, status, delivery_fee, product_name, price, stock_qty, is_prime_eligible, seller_id, seller_name, seller_email, seller_city, seller_rating, category_id, category_name, parent_category_id, quantity, unit_price	NEW	One row per item; order, product, seller and category data repeated
shipment	shipment_id	shipment_id, order_id, delivery_type, carrier, tracking_no, shipped_date, expected_date, delivered_date	NEW	Shipment group moved out
payment_flat	payment_id	payment_id, method_id, customer_id, method_type, provider, masked_detail, subscription_id, order_id, amount, payment_date, status	NEW	Payment-method details repeated
review	review_id	review_id, customer_id, product_id, rating, comment_text, review_date	NEW	Reviews group moved out
watch_history_flat	watch_id	watch_id, profile_id, content_id, episode_id, season_no, episode_no, episode_title, episode_duration, watched_at, progress_percent	NEW	Episode details repeated
watchlist_flat	(profile_id, content_id)	profile_id, content_id, title, genre, language, release_year, maturity_rating, rating, added_at	NEW	Content details repeated
music_play_flat	play_id	play_id, profile_id, track_id, track_title, album_name, duration_sec, release_year, artist_id, artist_name, artist_genre, played_at	NEW	Track and artist details repeated

Bold columns show the data that is still repeated. They are removed in 2NF and 3NF.

NEW = table created at this stage	CARRIED = table unchanged from the previous stage	KEPT = deliberate exception
--	--	------------------------------------

5. Second Normal Form (2NF): 17 tables

Rule: 1NF plus no partial dependency. In a table with a composite key, every non-key column must depend on the **whole** key. Only the three composite-key tables from 1NF needed fixing; tables with a single-column key are in 2NF automatically.

Partial dependencies found

1NF table	Composite key	Partial dependency	Split into
plan_benefit_flat	(plan_id, benefit_id)	benefit_id → benefit_name, benefit_description	plan_benefit + benefit
order_line_flat	(order_id, product_id)	order_id → order_date, status, delivery_fee, customer_id, address_id product_id → product_name, price, stock, seller_*, category_*	orders + product_flat + order_item
watchlist_flat	(profile_id, content_id)	content_id → title, genre, language, release_year, ...	watchlist + video_content

Tables present after 2NF

Table	Primary key	Columns / note	Status
customer	customer_id	Unchanged	CARRIED
address	address_id	Unchanged (single-column key)	CARRIED
profile	profile_id	Unchanged	CARRIED
subscription_flat	subscription_id	Unchanged, still holds plan columns (fixed in 3NF)	CARRIED
subscription_audit	audit_id	Unchanged	CARRIED
benefit	benefit_id	benefit_id, benefit_name, description	NEW
plan_benefit	(plan_id, benefit_id)	Only the two key columns remain	NEW
orders	order_id	order_id, customer_id, address_id, order_date, status, delivery_fee	NEW
order_item	(order_id, product_id)	order_id, product_id, quantity, unit_price	NEW
product_flat	product_id	product_id, product_name, price, stock_qty, is_prime_eligible + seller_* + category_* (fixed in 3NF)	NEW
shipment	shipment_id	Unchanged	CARRIED
payment_flat	payment_id	Unchanged, still holds method columns (fixed in 3NF)	CARRIED
review	review_id	Unchanged	CARRIED
watch_history_flat	watch_id	Unchanged, still holds episode columns (fixed in 3NF)	CARRIED
watchlist	(profile_id, content_id)	profile_id, content_id, added_at	NEW
video_content	content_id	content_id, title, content_type, genre, language, release_year, duration_min, maturity_rating, rating, is_prime_original	NEW
music_play_flat	play_id	Unchanged, still holds track and artist columns (fixed in 3NF)	CARRIED

NEW = table created at this stage

CARRIED = table unchanged from the previous stage

KEPT = deliberate exception

6. Third Normal Form (3NF): 24 tables

Rule: 2NF plus no transitive dependency (non-key → non-key). Five 2NF tables still hold data about another entity, so each determinant is moved into its own table and replaced by a foreign key.

2NF table	Transitive dependency	Split into
subscription_flat	plan_id → plan_name, price, duration_months, max_profiles, has_ads	plan + subscription
product_flat	seller_id → seller_name, email, city, rating category_id → category_name, parent_category_id	seller + category + product
payment_flat	method_id → customer_id, method_type, provider, masked_detail	payment_method + payment
watch_history_flat	episode_id → season_no, episode_no, episode_title, episode_duration	episode + watch_history
music_play_flat	track_id → track_title, album, duration, artist_id artist_id → artist_name, artist_genre	music_artist + music_track + music_play

Tables present after 3NF (final 24)

Module	Table	Primary key	Status
Account	customer	customer_id	CARRIED
	address	address_id	KEPT (A)
	profile	profile_id	CARRIED
Membership	plan	plan_id	NEW
	benefit	benefit_id	CARRIED
	plan_benefit	(plan_id, benefit_id)	CARRIED
	subscription	subscription_id	NEW
	subscription_audit	audit_id	CARRIED
	payment_method	method_id	NEW
Shopping	seller	seller_id	NEW
	category	category_id	NEW
	product	product_id	NEW
	orders	order_id	KEPT (B)
	order_item	(order_id, product_id)	CARRIED
	shipment	shipment_id	CARRIED
	payment	payment_id	NEW
	review	review_id	CARRIED
Video	video_content	content_id	CARRIED
	episode	episode_id	NEW
	watch_history	watch_id	KEPT (C)
	watchlist	(profile_id, content_id)	CARRIED
Music	music_artist	artist_id	NEW
	music_track	track_id	KEPT (D)
	music_play	play_id	NEW

KEPT = the table still contains one small transitive dependency that was left in on purpose (explained on the next page, A to D). Strictly speaking, these four tables are only in 2NF if that dependency is counted.

7. Boyce-Codd Normal Form (BCNF): same 24 tables, verified

Rule: for every functional dependency $X \rightarrow Y$, X must be a candidate key. No new table is needed. The check is that the only determinants are primary keys or UNIQUE columns.

Table	Determinants (keys)	Result	Table	Determinants (keys)	Result
customer	customer_id; email (UNIQUE)	BCNF	orders	order_id	Exc. B
address	address_id	Exc. A	order_item	(order_id, product_id)	BCNF
profile	profile_id; (customer_id, profile_name)	BCNF	shipment	shipment_id; order_id, tracking_no (UNIQUE)	BCNF
plan	plan_id; plan_name (UNIQUE)	BCNF	payment	payment_id	BCNF
benefit	benefit_id; benefit_name (UNIQUE)	BCNF	review	review_id; (customer_id, product_id)	BCNF
plan_benefit	(plan_id, benefit_id), all-key table	BCNF	video_content	content_id	BCNF
subscription	subscription_id	BCNF	episode	episode_id; (content_id, season_no, episode_no)	BCNF
subscription_audit	audit_id	BCNF	watch_history	watch_id	Exc. C
payment_method	method_id	BCNF	watchlist	(profile_id, content_id), all determinants are the key	BCNF
seller	seller_id; seller_name, email (UNIQUE)	BCNF	music_artist	artist_id; artist_name (UNIQUE)	BCNF
category	category_id; category_name (UNIQUE)	BCNF	music_track	track_id	Exc. D
product	product_id	BCNF	music_play	play_id	BCNF

Result: 20 tables are in BCNF; 4 keep one deliberate exception.

The four deliberate exceptions

Mark	Table	Dependency left in	Why it is kept
A	address	pincode $\rightarrow$ city, state	Keeps addresses simple. A pincode(pincode, city, state) table would remove it.
B	orders	address_id $\rightarrow$ customer_id	customer_id in orders makes joins and per-customer queries faster.
C	watch_history	episode_id $\rightarrow$ content_id	Movies have no episode row, so content_id is still needed.
D	music_track	album_name $\rightarrow$ release_year (if albums were entities)	The schema has no album table, so album is just an attribute.

Correct designs that are not violations

- **order_item.unit_price** is the price at purchase time and depends on (order_id, product_id), so past orders stay correct.
- **No total_amount in orders:** it is derived, so it is calculated in the view v_order_totals.
- **payment exclusive arc:** exactly one of subscription_id or order_id is filled, enforced by a CHECK constraint.

Summary

The unnormalized PRIME_MASTER table was normalized step by step into the final 24-table schema. **1NF** gives 13 tables by moving repeating groups into separate tables. **2NF** gives 17 tables by splitting the composite-key tables so that every column depends on the whole key. **3NF** gives 24 tables by extracting plan, seller, category, payment_method, episode, music_artist and music_track to remove transitive dependencies. **BCNF** keeps the same 24 tables because every determinant is already a key, apart from four deliberate exceptions (address, orders, watch_history and music_track) that are kept for simplicity and speed. Overall, 20 tables are fully BCNF and each piece of data is stored in one place.